

RoadVision AI

Intelligent Road Damage Detection and Analytics Using Deep Learning

Image detection and analytics dashboard

Python · YOLOv8n · RDD2022 · Google Colab · Streamlit

Prepared for the College GitHub workspace: techky-Kabilan

Repository: Deep-Learning-Lab

Project folder: mini-projects/roadvision-ai

Evidence status

Software implementation and automated checks are available. RDD2022 training has not been performed. Precision, recall and mAP are pending; no measured model results are claimed. This is an implementation report with a reproducible training protocol.

No institution-specific certificate, guide name, roll number or signed declaration is invented. Add verified academic identity details before formal submission.

Abstract and objectives

RoadVision AI is an educational image-based road-damage inspection system. A pretrained YOLOv8n detector is intended to be fine-tuned on the RDD2022 dataset to localize four target damage categories. A Streamlit interface accepts a JPG or PNG photograph and presents annotated bounding boxes, category labels, per-object confidence, class counts and a confidence distribution. Annotated PNG and CSV downloads make the inspection record portable.

The project connects dataset preparation, transfer learning, reproducible evaluation and a usable interface. It limits scope to photographs; video tracking, geolocation, severity estimation and automated maintenance decisions are excluded. Predictions must be reviewed by a person.

Objectives

1. Convert official VOC annotations to normalized YOLO labels.
2. Preserve reproducible training, validation and held-out test partitions.
3. Fine-tune a resource-efficient pretrained detector in Colab.
4. Measure localization-aware performance using per-class metrics and confusion matrices.
5. Deliver a dashboard and exportable visual evidence.

Report map

02 Problem and scope · 03 Dataset and licensing · 04 System architecture · 05 Training methodology · 06 Evaluation protocol · 07 Interface evidence · 08 Verification and reproducibility · 09 Limitations and next steps · 10 References.

Problem definition and project scope

Road defects vary in geometry and appearance. A single photograph can contain multiple crack types and potholes. Whole-image classification cannot identify the location of each defect, so object detection is the appropriate formulation. The goal is to assist review of visible defects, not to certify a road as safe.

Requirement	Implemented behavior
Input	One JPG/PNG photograph; 20 MB and 25 megapixel limits; RGB conversion and EXIF correction
Detection	Trusted fine-tuned checkpoint; class mapping validated before inference
Evidence	Boxes, dataset class codes, labels and confidence scores
Analytics	Current-image object counts for all four classes and confidence histogram
Export	Annotated PNG and per-object CSV; header-only CSV for zero detections
Excluded	Video, severity labels, GPS mapping, historical storage and pavement condition scoring

Confidence is a model score. It is not calibrated probability, physical damage extent or hazard severity. Counts summarize detected boxes; they do not estimate damaged road length. A threshold change can alter both counts and false-positive/false-negative behavior.

Dataset, classes and licensing

RDD2022 is a multi-national road-damage dataset described by Arya et al. The official repository specifies the four target categories below. Labelled training images have Pascal VOC XML annotations; the official challenge test images do not have public XML ground truth. [1,2]

YOLO ID	Code	Category
0	D00	Longitudinal crack
1	D10	Transverse crack
2	D20	Alligator crack
3	D40	Pothole

The Colab notebook starts with the India subset to reduce storage and training demands. Any report of a completed run must state the actual countries and image/class support. Other XML categories are explicitly ignored and counted in the preparation summary. Exact byte-identical images are deduplicated before splitting.

Source availability

The official India S3 archive returned HTTP 403 during project preparation. The notebook includes a Drive archive fallback and points to the official Figshare record; it does not silently use an unverified mirror. The full Figshare archive is approximately 13.26 GB, substantially larger than the India subset. [1,2]

Licensing discrepancy

The official repository README describes dataset images as CC BY-SA 4.0, while the RDD2022 Figshare record displays CC BY 4.0. Resolve the terms of the actual downloaded artifact before redistribution. Repository code licences do not automatically cover dataset images. No dataset, archive or trained weights are bundled. [1,2]

System architecture and implementation

Stage	Responsibility
Data preparation	Read XML safely; filter classes; normalize coordinates; remove exact duplicates; save split manifest
Training	Load pretrained YOLOv8n; fine-tune on training data; choose checkpoint using validation
Evaluation	Score held-out labelled test data; export metrics, per-class CSV, PR curves and confusion matrices
Image input	Decode only JPEG/PNG; reject malformed or oversized inputs; correct orientation
Inference	Validate checkpoint labels; resize/letterbox through YOLO; apply confidence filtering and NMS
Presentation	Render annotations, counts and distribution; export PNG and CSV

Python modules separate data validation and export logic from the model adapter and Streamlit interface. The model is cached as a resource and inference is protected by a lock. Results are bound to the image bytes, thresholds and checkpoint modification time; changing an input requires another analysis.

The application never substitutes a general COCO checkpoint for a road-damage model. Without `best.pt` it offers upload preview and clearly states that training is pending. It checks the exact class order 0:D00, 1:D10, 2:D20, 3:D40 before inference. Only trusted checkpoints should be loaded because serialized `.pt` files can contain executable content.

The interface uses a restrained product layout with a fixed type hierarchy, visible focus indicators, contrast-conscious blue/neutral colors, responsive columns and short press feedback. Reduced-motion preference disables animation. It avoids decorative dashboard movement.

Training methodology and reproducibility

Transfer learning

YOLOv8n provides an established pretrained detector with a small model footprint. Fine-tuning adapts generic image features to RDD2022 classes. This choice supports a feasible college project; it does not imply optimal accuracy. [3]

Setting	Planned starting value
Model	Pretrained yolov8n.pt
Input size	640 pixels
Epoch limit	50; early stopping patience 10
Batch	16; lower if GPU memory is insufficient
Seed	42; deterministic mode requested
Partitions	70/15/15 of independent groups; actual image ratios vary
Runtime	Google Colab GPU; local CPU app inference supported

VOC 1-based inclusive coordinates are converted to continuous pixel extents, clamped to the image and normalized as center-x, center-y, width and height. Invalid boxes fail preparation. Output folders refuse overwrite. Images with no target objects retain empty label files and serve as background examples.

A JSON route/sequence manifest can group related images before splitting. Without it, each unique image is an independent group and adjacent scenes may remain correlated across splits. Exact deduplication does not solve near-duplicate leakage. Audit route overlap, class support and country distribution before making generalization claims.

Save the split manifest, support counts, training arguments, checkpoint SHA-256, environment, commit and epochs actually completed. Colab sessions can disconnect; completed artifacts are copied to Drive after each major stage. A seed does not guarantee bit-identical GPU results.

Meaningful model evaluation

Final scored evaluation uses the held-out labelled partition, not the unlabelled official challenge test set. Hyperparameters and thresholds are chosen on validation data. Test results must be reported once from the selected checkpoint, with per-class support and qualitative failures. [1,4]

Metric	Interpretation	Current result
Precision	TP / (TP + FP); reliability of predicted objects	Pending
Recall	TP / (TP + FN); coverage of annotated objects	Pending
mAP@0.5	Class-average AP at IoU 0.5	Pending
mAP@0.5:0.95	AP averaged across IoU 0.5 to 0.95	Pending
Confusion matrix	Class confusions plus unmatched/background cases	Pending

A true-positive detection requires the correct class and adequate overlap with a ground-truth box under one-to-one matching. IoU is intersection area divided by union area. AP summarizes the precision-recall ranking; averaging AP across categories yields mAP. Stricter overlap thresholds test localization quality.

The evaluator uses Ultralytics validation with confidence 0.001, IoU setting 0.6 and plots enabled. Low confidence retains candidates for AP ranking; dashboard inference defaults to 0.25. Summary precision/recall use the library operating point. The evaluator exports raw and normalized detection confusion matrices, PR curves, overall metrics, per-class values and provenance. Read matrix axis labels and background cells before interpreting errors. [4]

No measured results are present. Software tests and synthetic interface fixtures do not measure road-damage accuracy. A completed experiment must add actual values, sample sizes, support counts, confidence operating point and failure examples without replacing pending values with illustrative numbers.

Interface screenshots and export workflow

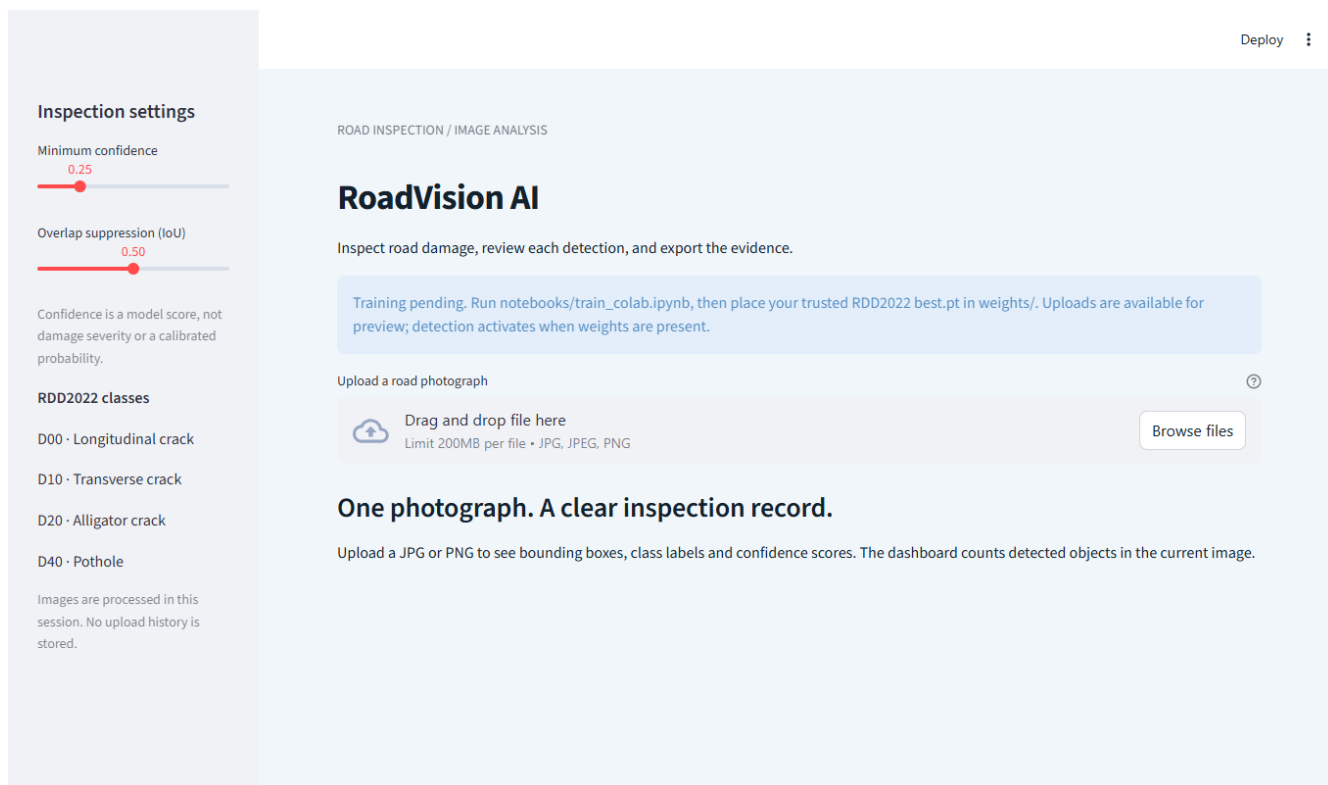


Figure 1. Actual app startup without a trained checkpoint.

Inspection evidence and analytics

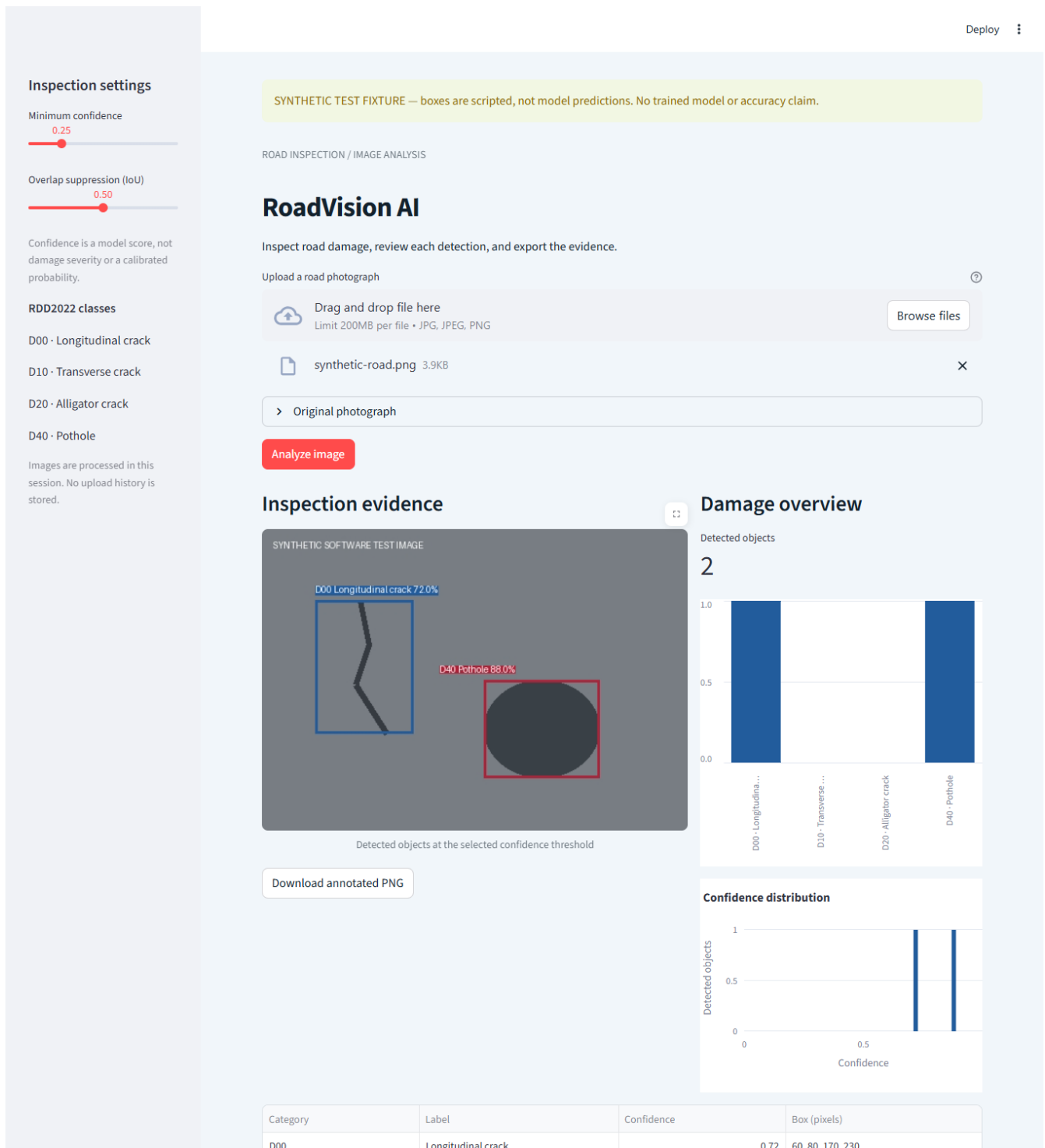


Figure 2. App workflow exercised with explicitly labelled synthetic test detections. No YOLO model produced these boxes.

Upload a photograph, set thresholds and click Analyze image when trusted weights are installed. Review annotated objects, count bars and confidence distribution. Download the PNG for visual evidence and CSV for tabular inspection. Zero detections produces a clear message and an empty export with a header. Real-model screenshots remain pending.

Verification and repository delivery

Automated tests cover real image decoding, format rejection, empty and populated CSV export, formula-safe filenames, annotation changes, invalid boxes/scores, checkpoint mapping rejection, XML normalization, deterministic partitions, no-overwrite behavior, sequence isolation and Streamlit startup without weights.

The final verified suite passed 13 tests. Additional workflow checks exercise model output adaptation, evaluator artifact/provenance export and browser exports using synthetic fixtures. These checks establish software behavior only; pretrained-model execution, GPU training and measured generalization are separate evidence requirements.

The project is isolated inside Deep-Learning-Lab under mini-projects/roadvision-ai. Existing laboratory experiments are not overwritten. A path-filtered GitHub Actions workflow runs the project tests with Python 3.12. Dependencies are pinned for the main application stack. The code licence applies only to this mini-project folder.

Local operation

Create a Python 3.12 virtual environment, install requirements-dev.txt, run `python -m pytest -q`, then `streamlit run app.py`. After training, place trusted best.pt in weights/. Use `python -m scripts.evaluate` with the dataset YAML to produce final test artifacts.

Dataset images, archives, weights and generated run folders are ignored by Git. The checked-in report, documentation and labelled screenshots are small portable artifacts. Distribute checkpoints through external model hosting or a release only after size, licensing and provenance review.

Limitations, future work and conclusion

The implementation is ready for a documented training experiment, but a trained road-damage detector has not been delivered. Thin cracks, shadows, painted markings, compression, rain, camera angle and unseen countries can affect detections. Class imbalance and annotation ambiguity can conceal weaknesses in aggregate metrics.

Image-level splitting may overestimate generalization when related driving scenes cross partitions. Use route metadata, near-duplicate auditing and country-specific evaluation where available. Confidence does not measure severity. A photograph has no reliable scale for estimating crack width or repair priority without additional calibration.

Completion checklist

1. Obtain the official dataset and verify artifact-specific terms.
2. Review partitions and class support; supply route groups where possible.
3. Execute Colab training and retain run evidence.
4. Evaluate the selected checkpoint on held-out labelled data.
5. Inspect false positives/negatives and latency on the intended device.
6. Update this report and model card from actual evidence.
7. Capture real inference screenshots and conduct a college demonstration.

Possible future work includes country-held-out evaluation, stronger sequence splitting, calibration studies and comparison with a larger YOLO variant. Video, GIS and severity prediction remain future extensions requiring new requirements and validation.

RoadVision AI provides a coherent mini-project pipeline from official annotations to a human-reviewable dashboard and exports. Its current contribution is the implemented workflow and reproducible evaluation protocol. Claims about model quality await verified training and evaluation.

References and provenance

[1] Official dataset repository and category definitions

<https://github.com/sekilab/RoadDamageDetector>

[2] Arya et al. RDD2022 Figshare dataset, DOI 10.6084/m9.figshare.21431547

<https://figshare.com/articles/dataset/21431547>

[3] Ultralytics YOLOv8 model documentation

<https://docs.ultralytics.com/models/yolov8/>

[4] Ultralytics model validation and detection metrics

<https://docs.ultralytics.com/modes/val/>

[5] Ultralytics licensing

<https://www.ultralytics.com/license>

[6] Streamlit documentation

<https://docs.streamlit.io/>

[7] CC BY-SA 4.0 terms

<https://creativecommons.org/licenses/by-sa/4.0/>

[8] CC BY 4.0 terms

<https://creativecommons.org/licenses/by/4.0/>

Dataset article: Arya, D.; Maeda, H.; Ghosh, S. K.; Toshniwal, D.; Sekimoto, Y. RDD2022: A multi-national image dataset for automatic road damage detection. *Geoscience Data Journal*, 11(4), 846-862, 2024. Bibliographic details follow the official repository.

Sources checked on 09 October 2026. Dataset download availability and licence labels are recorded as observed, not guarantees of future availability. No external benchmark values are copied as project results.